DAY 5 · C PROGRAMMING SERIES

C Programming Operators

The Complete Guide

From historical origins on the PDP-11 to advanced bitwise implementation — plus three classic interview problems solved entirely with operators.

Part 1 — Historical Background & Evolution

1.1 The Origins of C Operators

The BCPL and B Heritage

C's operator set traces back to Martin Richards' BCPL (1966) and Ken Thompson's B language (1969). The operators were designed to closely map to PDP-11 machine instructions.

```
Historical Evolution:
BCPL (1966) → B (1969) → C (1972) → ANSI C (1989) → C99 / C11 / C18
```

Why this matters: understanding this lineage explains why C operators sit so close to the hardware — they were literally designed to compile to single machine instructions on the PDP-11.

The PDP-11 Architecture Influence

The PDP-11 had:

- 16-bit word size
- Direct bit manipulation instructions
- Register-based operations
- Efficient increment/decrement addressing modes

This directly influenced:

- Bitwise operators (`&` , `|` , `^` , `~` , `<<` , `>>`)
- Increment/decrement operators (`++` , `--`)
- Compound assignments (`+=` , `-=` , etc.)

1.2 Why C Operators are "Low-Level"

```
C Philosophy: "Trust the programmer"
- Operators map directly to CPU instructions
- No implicit safety checks (unlike Python/Java)
- Maximum performance at the cost of potential errors
- Ideal for system programming
```

Performance comparison:

```
// High-level abstraction (Java/Python)
Integer sum = a + b;        // May involve object creation, bounds checking

// Low-level C (maps to a single ADD instruction)
int sum = a + b;            // Single CPU cycle on most architectures
```

Part 2 — Theoretical Foundations

2.1 Operator Theory in Computer Science

Formal Definition

An operator is a function that maps operands to a result, where operands are values and the result is a value. In C, this is implemented through the operator grammar.

```
Operator Grammar:
Expression ::= Term {Operator Term}
Term       ::= Primary | Unary_Operator Primary
Primary    ::= Constant | Variable | (Expression)
```

Operator Overloading Limitations

Unlike C++, C does **not** support operator overloading:

```c
// C: Operators only work on built-in types
int a = 5, b = 10;
int c = a + b;        // OK - integers

struct Complex x, y;
// z = x + y;         // ERROR! Cannot overload + for structs
```

2.2 Type Systems and Operators

Type Promotion Rules

When operands have different types, C performs implicit type conversion:

```c
char + int     → int          // Integer promotion
short + int    → int          // Integer promotion
int + long     → long         // Long promotion
float + double → double       // Double promotion
```

The "usual arithmetic conversions":

```
Step 1: long double ← float / double
Step 2: double      ← float
Step 3: float       ← (float remains float)
Step 4: Integer promotions (char/short → int)
Step 5: unsigned long ← long ← unsigned int ← int
```

Implementation Example

```c
#include <stdio.h>
int main() {
    char c = 100;            // 8-bit signed
    short s = 1000;          // 16-bit signed
    int i = 100000;          // 32-bit signed
    float f = 3.14;          // 32-bit float

    // Type promotion in action
    float result = c + s + i + f;
    // All promoted to float before calculation

    printf("Result: %f\n", result);
    printf("Size of expression: %lu\n", sizeof(c + s + i + f));
    // Outputs: 4 (float size)
    return 0;
}
```

2.3 L-values and R-values

L-value (Location Value): an expression that refers to a memory location. **R-value** (Read Value): an expression that yields a value, not an address.

```c
int x = 5;           // x = l-value, 5 = r-value
x = x + 1;            // x (LHS) = l-value, x + 1 (RHS) = r-value

int* ptr = &x;         // &x = r-value (address), ptr = l-value

&x = 5;                // ERROR! &x is an r-value
```

> **Important:** the assignment operator `=` requires an l-value on the left side.

Part 3 — Deep Dive Into Each Operator Category

3.1 Arithmetic Operators — Advanced Concepts

Integer Division and Rounding

```c
// C99 Standard: division truncates toward zero
#include <stdio.h>
int main() {
    printf("%d\n", 7 / 3);    // 2  (truncate toward zero)
    printf("%d\n", -7 / 3);   // -2 (truncate toward zero)
    printf("%d\n", 7 / -3);   // -2 (truncate toward zero)
    return 0;
}
```

Modulo Operation — The Mathematical Foundation

```
Definition: a % b = a - (a / b) * b

Properties:
1. (a + b) % c = ((a % c) + (b % c)) % c
2. (a * b) % c = ((a % c) * (b % c)) % c
3. a % b = a - b * floor(a / b)   [for the mathematical mod]
```

C implementation quirks:

```c
int a = -5, b = 3;
printf("%d\n", a % b);    // -2 (not 1 as in mathematics)
// C follows: a % b = a - (a/b)*b
// -5 - (-1)*3 = -2
```

Overflow Detection

```c
#include <limits.h>
#include <stdbool.h>

bool safe_add(int a, int b, int* result) {
    if (b > 0 && a > INT_MAX - b) return false;  // Positive overflow
    if (b < 0 && a < INT_MIN - b) return false;  // Negative overflow
    *result = a + b;
    return true;
}

int main() {
    int x = 2147483647;
    int y = 1;
    int result;

    if (!safe_add(x, y, &result)) {
        printf("Overflow detected!\n");
    }
    return 0;
}
```

3.2 Bitwise Operators — Deep Implementation

Binary Representation and Two's Complement

```
Two's Complement for -5 (8-bit):
Step 1: +5          = 00000101
Step 2: Invert bits  = 11111010
Step 3: Add 1        = 11111011
Result: 11111011 = -5
```

Bitwise NOT (`~`) in Action

```c
#include <stdio.h>
#include <limits.h>

int main() {
    unsigned int x = 5;      // 00000000 00000000 00000000 00000101
    unsigned int y = ~x;     // 11111111 11111111 11111111 11111010
    printf("%u\n", y);       // 4294967290 (2^32 - 6)

    signed int z = 5;        // 00000000 00000000 00000000 00000101
    signed int w = ~z;       // 11111111 11111111 11111111 11111010
    printf("%d\n", w);       // -6 (two's complement)
    return 0;
}
```

Shift Operations — Deep Dive

Left shift (`<<`)

```
x << n = x * 2^n   (for non-negative x, no overflow)
Implementation: moves bits left, fills with 0s
```

Right shift (`>>`) — two types

```c
// 1. Logical right shift (unsigned)
unsigned int x = 0x80000000;  // 10000000...
unsigned int y = x >> 1;      // 01000000... (fills with 0)

// 2. Arithmetic right shift (signed)
signed int z = -1;            // 11111111...
signed int w = z >> 1;        // 11111111... (fills with 1)
```

Implementation example:

```c
#include <stdio.h>
void print_bits(int n, int bits) {
    for (int i = bits - 1; i >= 0; i--) {
        printf("%d", (n >> i) & 1);
    }
    printf("\n");
}

int main() {
    int x = 16;
    printf("x = 16: ");
    print_bits(x, 8);        // 00010000

    printf("x << 2: ");
    print_bits(x << 2, 8);   // 01000000 (multiply by 4)

    printf("x >> 2: ");
    print_bits(x >> 2, 8);   // 00000100 (divide by 4)

    // Signed right shift
    signed int y = -16;
    printf("y = -16: ");
    print_bits(y, 8);        // 11110000

    printf("y >> 2: ");
    print_bits(y >> 2, 8);   // 11111100 (divide by 4, preserve sign)
    return 0;
}
```

Bit Manipulation Techniques

1. Isolate lowest set bit

```
int lowest_bit = x & -x;   // -x = (~x + 1)
// If x = 12 (1100), lowest_bit = 4 (0100)
```

2. Isolate highest set bit

```
int highest_bit;
for (highest_bit = 1 << 31; !(x & highest_bit); highest_bit >>= 1);
// This loops until the highest set bit is found
```

3. Count set bits (Brian Kernighan's algorithm)

```
int count_bits(unsigned int x) {
    int count = 0;
    while (x) {
        x &= (x - 1);   // Clear lowest set bit
        count++;
    }
    return count;
}
// x = 1011 (11) → 1011 & 1010 = 1010 (10) → count:1
//                 1010 & 1001 = 1000 (8)  → count:2
//                 1000 & 0111 = 0000 (0)  → count:3
```

4. Reverse bits

```
unsigned int reverse_bits(unsigned int x) {
    unsigned int result = 0;
    for (int i = 0; i < 32; i++) {
        result <<= 1;
        result |= x & 1;
        x >>= 1;
    }
    return result;
}
```

3.3 Logical Operators — Short-Circuit Deep Dive

Short-Circuit Evaluation Implementation

```
Expression: A && B
Evaluation:
1. Evaluate A
2. If A == false, return false (B is NOT evaluated)
3. If A == true, evaluate B
4. Return the result of B
```

Practical Applications of Short-Circuit

1. Null pointer safety

```
if (ptr != NULL && ptr->data == 5) {
    // Safe! ptr is only dereferenced if not NULL
}
```

2. Array bounds checking

```c
if (index >= 0 && index < size && array[index] == value) {
    // Safe! array is only accessed if index is valid
}
```

3. Lazy initialization

```c
int initialized = 0;
int cache;
if (!initialized && (cache = compute_expensive()) > 0) {
    // compute_expensive() is only called if needed
    initialized = 1;
}
```

Short-Circuit in Conditionals

```c
// Common C idiom: if success, then report
int success = operation();
success && printf("Operation succeeded\n");
// Equivalent: if (success) printf(...)

// Similar for OR
int error = operation();
error || printf("No error\n");
// Equivalent: if (!error) printf(...)
```

3.4 Assignment Operators — Memory Model

How Assignment Works in Memory

```c
int a = 5;
int b = a;      // Memory copy: value copied from a's location to b's location

int* ptr = &a;  // ptr stores the address of a
```

Memory diagram:

Memory Address	Value
0x1000 (a)	5
0x1004 (b)	5
0x1008 (ptr)	0x1000 (address of a)

Compound Assignment Performance

```c
// These are equivalent, but the right one is more efficient:
a = a + b;    // May load 'a' twice
a += b;       // Loads 'a' once, modifies in place
```

Assembly comparison (x86_64):

```
; a = a + b
mov eax, [a]     ; Load a
add eax, [b]     ; Add b
mov [a], eax     ; Store back

; a += b
add [a], [b]     ; Direct memory modification (fewer instructions)
```

Part 4 — Advanced Operator Concepts

4.1 Operator Precedence and Parse Trees

Mathematical Foundation

Expressions form a parse tree based on precedence and associativity:

```
Expression: a * b + c / d

Parse Tree:
      +
     / \
    *   /
   / \ / \
  a  b c  d

Evaluation order follows tree traversal (post-order):
1. Evaluate a * b
2. Evaluate c / d
3. Add the two results
```

Parsing Implementation

```c
// Simplified expression parser for + and *
// Precedence: * > +
// Left associativity for both

#include <stdio.h>
#include <ctype.h>

char* expr;

int parse_add_sub();
int parse_mul_div();

int parse_number() {
    int num = 0;
    while (isdigit(*expr)) {
        num = num * 10 + (*expr - '0');
        expr++;
    }
    return num;
}

int parse_mul_div() {
    int left = parse_number();
    while (*expr == '*' || *expr == '/') {
        char op = *expr++;
        int right = parse_number();
        if (op == '*') left *= right;
        else left /= right;
    }
    return left;
}

int parse_add_sub() {
    int left = parse_mul_div();
    while (*expr == '+' || *expr == '-') {
        char op = *expr++;
        int right = parse_mul_div();
        if (op == '+') left += right;
        else left -= right;
    }
    return left;
}
```

4.2 Side Effects and Sequence Points

Definition

A **sequence point** is a point in program execution where all side effects of previous operations are guaranteed to be completed.

Sequence points in C occur:

- At the end of full expressions (semicolon)
- At `||`, `&&`, `?:`, and `,` operators
- At function call entry and return
- At the end of a full declarator

Undefined Behavior Examples

```c
// ALL UNDEFINED BEHAVIOR:

int i = 0;
i = i++;                    // Undefined (two writes)
a[i] = i++;                 // Undefined (index and increment)
i = ++i + i;                // Undefined (multiple modifications)

// Function arguments
printf("%d %d", i++, i++);  // Undefined (order unspecified)

// More subtle
int arr[10];
int x = 5;
arr[x] = x++;               // Undefined
```

Avoiding Side Effect Issues

```c
// Correct approach:
int i = 0;
i = i + 1;              // Well-defined

// Or:
++i;                     // Well-defined (single operation)

// Function calls:
int i = 5;
printf("%d %d\n", i, i++);   // UNDEFINED
printf("%d %d\n", i++, i);   // UNDEFINED

// Correct:
int i = 5;
int temp = i++;
printf("%d %d\n", temp, i);  // Well-defined
```

4.3 The Comma Operator

Theory

The comma operator `,` evaluates both operands and returns the value of the right operand.

```c
int x = (5, 10);   // x = 10
```

Practical Uses

```
// 1. In for loops
for (int i = 0, j = 10; i < 10; i++, j--) {
    // Multiple initializations and increments
}

// 2. Debugging
int result = (debug_function(), compute_value());

// 3. Macros
#define SWAP(a, b) (a ^= b, b ^= a, a ^= b)

// 4. Conditional assignment
if (x > y)
    max = x, min = y;   // Two statements in one line
else
    max = y, min = x;
```

Part 5 — Application in Embedded Systems

5.1 Register-Level Programming

```
// Microcontroller register manipulation
#define GPIO_PORT  0x40021000
#define SET_BIT(reg, bit)    ((reg) |= (1 << (bit)))
#define CLEAR_BIT(reg, bit)  ((reg) &= ~(1 << (bit)))
#define TOGGLE_BIT(reg, bit) ((reg) ^= (1 << (bit)))
#define CHECK_BIT(reg, bit)  (((reg) >> (bit)) & 1)

// Real-world STM32 GPIO configuration
volatile uint32_t* GPIOA_MODER = (uint32_t*)0x40020000;
volatile uint32_t* GPIOA_ODR   = (uint32_t*)0x40020014;

// Configure PIN 5 as output (01 in MODER)
*GPIOA_MODER &= ~(3 << (5 * 2));  // Clear bits
*GPIOA_MODER |= (1 << (5 * 2));   // Set to output mode

// Toggle LED
*GPIOA_ODR ^= (1 << 5);
```

5.2 Bitfields in Embedded Programming

```
// Using bitfields (but careful with endianness)
struct StatusRegister {
    unsigned int enable   : 1;  // Bit 0
    unsigned int mode     : 2;  // Bits 1-2
    unsigned int speed    : 2;  // Bits 3-4
    unsigned int reserved : 3;  // Bits 5-7
    unsigned int error    : 1;  // Bit 8
};

// Same using bitwise operators (more portable)
#define STATUS_ENABLE_MASK   0x001
#define STATUS_MODE_MASK     0x006
#define STATUS_SPEED_MASK    0x018
#define STATUS_ERROR_MASK    0x100

#define SET_STATUS_ENABLE(reg)   ((reg) |= STATUS_ENABLE_MASK)
#define CLR_STATUS_ENABLE(reg)   ((reg) &= ~STATUS_ENABLE_MASK)
#define GET_STATUS_MODE(reg)     (((reg) & STATUS_MODE_MASK) >> 1)
```

5.3 Efficient Multiplication/Division by Powers of Two

```
// Slow (requires division)
int x = y / 8;
int x = y * 32;

// Fast (shift operations)
int x = y >> 3;   // divide by 8
int x = y << 5;   // multiply by 32

// Note: only works for unsigned values, or when sign doesn't matter
```

5.4 CRC Calculation Example

```c
#include <stdint.h>

uint32_t crc32_bitwise(uint32_t crc, uint8_t data) {
    crc ^= data;
    for (int i = 0; i < 8; i++) {
        if (crc & 1)
            crc = (crc >> 1) ^ 0xEDB88320;
        else
            crc >>= 1;
    }
    return crc;
}

// Using a table for speed (common in embedded)
uint32_t crc32_table[256];

void init_crc_table() {
    for (int i = 0; i < 256; i++) {
        uint32_t crc = i;
        for (int j = 0; j < 8; j++) {
            if (crc & 1)
                crc = (crc >> 1) ^ 0xEDB88320;
            else
                crc >>= 1;
        }
        crc32_table[i] = crc;
    }
}
```

Part 6 — Performance Optimization

6.1 Optimization Techniques

```c
// 1. Using bitwise operations for faster modulo by a power of 2
int x = y & 7;           // y % 8
int x = y & 31;          // y % 32

// 2. Fast absolute value
int abs_val = (x ^ (x >> 31)) - (x >> 31);

// 3. Check if two numbers have the same sign
bool same_sign = !((a ^ b) >> 31);

// 4. Min/Max without branching
int min = y ^ ((x ^ y) & -(x < y));
int max = x ^ ((x ^ y) & -(x < y));

// 5. Power of 2 check
bool is_power_of_two = x && !(x & (x - 1));
```

6.2 Assembly Output Analysis

```c
// Example: comparing different implementations
// gcc -S -O2 program.c

int main() {
    int a = 5, b = 3;

    // Multiplication
    int c = a * 8;      // Compiled to: lea eax, [rdi*8]

    // Division
    int d = a / 2;      // Compiled to: sar eax, 1

    // Modulo
    int e = a % 16;     // Compiled to: and eax, 15

    return c + d + e;
}
```

Part 7 — Common Pitfalls and Debugging

7.1 Operator Precedence Table (Detailed)

Precedence	Operators	Associativity	Explanation
1	`()` `[]` `->` `.`	L→R	Function call, array, member access
2	`!` `~` `++` `--` `+` `-` `*` `&` `(type)` `sizeof`	R→L	Unary operators
3	`*` `/` `%`	L→R	Multiplicative
4	`+` `-`	L→R	Additive
5	`<<` `>>`	L→R	Shift
6	`<` `<=` `>` `>=`	L→R	Relational
7	`==` `!=`	L→R	Equality
8	`&`	L→R	Bitwise AND
9	`^`	L→R	Bitwise XOR
10	`|`	L→R	Bitwise OR
11	`&&`	L→R	Logical AND
12	`||`	L→R	Logical OR
13	`?:`	R→L	Conditional
14	`=` `+=` `-=` `*=` `/=` `%=` `&=` `^=` `|=` `<<=` `>>=`	R→L	Assignment
15	`,`	L→R	Comma

7.2 Debugging Checklist

```c
// Common bugs and fixes

// Bug 1: Assignment vs Comparison
if (flag = 1) { }           //  Always true
if (flag == 1) { }          //  Correct

// Bug 2: Bitwise vs Logical
if (a & b) { }              //  Bitwise, use && for logical
if (a && b) { }             //  Logical AND

// Bug 3: Shift Precedence
a = b << 2 + 3;             //  Evaluates as b << (2 + 3)
a = (b << 2) + 3;           //  Clear intent

// Bug 4: Signed vs Unsigned
int x = -5;
unsigned int y = 10;
if (x > y) { }              //  x promoted to unsigned → 4294967291
if ((unsigned)x > y) { }    //  Still wrong
if (x > (int)y) { }         //  Compare as signed

// Bug 5: Order of Evaluation
x = (y = 5) + (y = 3);      //  Undefined
x = 5; y = 3; z = x + y;    //  Well-defined

// Bug 6: Division by Zero
int result = 10 / a;        //  If a is 0
if (a != 0) result = 10/a;  //  Safe
```

7.3 Static Analysis Tools

```c
// Using GCC warnings
// gcc -Wall -Wextra -Wshadow -Wconversion program.c

int main() {
    int x = 5;
    int y = 3;

    // Warning: comparison between signed and unsigned
    unsigned int z = 10;
    if (x < z) { }   // -Wsign-compare warning

    // Warning: implicit conversion changes value
    char c = 300;    // -Wconversion warning

    // Warning: unused variable
    int unused;      // -Wunused-variable warning

    return 0;
}
```

Part 8 — Advanced Exercises

Exercise 1: Gray Code Conversion

Gray code is a binary reflected code commonly used in rotary encoders.

```c
unsigned int binary_to_gray(unsigned int n) {
    return n ^ (n >> 1);
}

unsigned int gray_to_binary(unsigned int n) {
    unsigned int mask;
    for (mask = n >> 1; mask != 0; mask = mask >> 1) {
        n = n ^ mask;
    }
    return n;
}

// Alternative efficient method
unsigned int gray_to_binary_fast(unsigned int n) {
    unsigned int mask = n;
    while (mask >>= 1) {
        n ^= mask;
    }
    return n;
}
```

Exercise 2: Circular Buffer Implementation

```c
#define BUFFER_SIZE 16
#define BUFFER_MASK (BUFFER_SIZE - 1)

typedef struct {
    int data[BUFFER_SIZE];
    unsigned int head;
    unsigned int tail;
} CircularBuffer;

// Use bitwise AND for modulo operation (faster)
void push(CircularBuffer* cb, int value) {
    cb->data[cb->head & BUFFER_MASK] = value;
    cb->head = (cb->head + 1) & BUFFER_MASK;
}

int pop(CircularBuffer* cb) {
    int value = cb->data[cb->tail & BUFFER_MASK];
    cb->tail = (cb->tail + 1) & BUFFER_MASK;
    return value;
}

// Buffer full check using bitwise XOR
int is_full(CircularBuffer* cb) {
    return ((cb->head ^ cb->tail) == BUFFER_SIZE);
}
```

Exercise 3: Bit Reversal Using Divide and Conquer

```c
// Fast bit reversal using swaps
unsigned int reverse_bits_dac(unsigned int x) {
    x = ((x & 0x55555555) << 1) | ((x & 0xAAAAAAAA) >> 1);
    x = ((x & 0x33333333) << 2) | ((x & 0xCCCCCCCC) >> 2);
    x = ((x & 0x0F0F0F0F) << 4) | ((x & 0xF0F0F0F0) >> 4);
    x = ((x & 0x00FF00FF) << 8) | ((x & 0xFF00FF00) >> 8);
    x = (x << 16) | (x >> 16);
    return x;
}
// Example: 0x12345678 → 0x1E6A2C48
```

Exercise 4: Endianness Detection

```c
#include <stdio.h>

// Using bitwise operations
int is_little_endian() {
    int x = 1;
    return *(char*)&x;   // Returns 1 for little-endian
}

// Using a union
union EndianCheck {
    int i;
    char c[4];
};

int check_endianness() {
    union EndianCheck ec = { .i = 1 };
    return ec.c[0] == 1;   // 1 = little endian, 0 = big endian
}

// Convert between endianness
unsigned int swap_endian(unsigned int x) {
    return ((x & 0x000000FF) << 24) |
           ((x & 0x0000FF00) << 8)  |
           ((x & 0x00FF0000) >> 8)  |
           ((x & 0xFF000000) >> 24);
}
```

Part 9 — Real-World Case Studies

Case Study 1: Embedded Sensor Reading

```c
// Reading a 12-bit ADC value from a 16-bit register
#define ADC_REGISTER 0x40021000
#define ADC_SIGN_BIT 11

int read_adc(void) {
    volatile uint32_t* adc = (uint32_t*)ADC_REGISTER;
    uint32_t raw = *adc;

    // Extract 12-bit value from bits 15-4
    int value = (raw >> 4) & 0xFFF;

    // Check if value is signed (bit 11)
    if (value & (1 << ADC_SIGN_BIT)) {
        value |= ~((1 << ADC_SIGN_BIT) - 1);   // Sign extend
    }

    return value;
}
```

Case Study 2: Simple Encryption (XOR Cipher)

```c
void xor_cipher(char* data, size_t len, unsigned char key) {
    for (size_t i = 0; i < len; i++) {
        data[i] ^= key;
        key = (key << 1) | (key >> 7);   // Rotate key
    }
}

// Example usage
char message[] = "Hello World!";
unsigned char key = 0x55;
xor_cipher(message, strlen(message), key);
printf("Encrypted: %s\n", message);
xor_cipher(message, strlen(message), key);
printf("Decrypted: %s\n", message);
```

Case Study 3: Error Correction — Hamming Code

```c
unsigned char compute_parity(unsigned char data) {
    // For 4-bit data + 3 parity bits
    unsigned char p1 = (data & 0x01) ^ (data & 0x02) ^ (data & 0x04);
    unsigned char p2 = (data & 0x01) ^ (data & 0x02) ^ (data & 0x08);
    unsigned char p4 = (data & 0x01) ^ (data & 0x04) ^ (data & 0x08);

    return (p1 << 1) | (p2 << 2) | (p4 << 4);
}
```

Part 10 — Comprehensive Summary

Operator Category Summary

Category	Operators	Count	Common Use
Arithmetic	`+` `-` `*` `/` `%`	5	Math calculations
Increment/Decrement	`++` `--`	2	Loop counters
Relational	`==` `!=` `<` `>` `<=` `>=`	6	Comparisons
Logical	`&&` `||` `!`	3	Conditions
Bitwise	`&` `|` `^` `~` `<<` `>>`	6	Bit manipulation
Assignment	`=` `+=` `-=` `*=` `/=` `%=` (+more)	11	Variable assignment
Other	`?:` `,` `sizeof` `&` `*`	6	Miscellaneous

Key Takeaways for Embedded Development

1. **Efficiency** — bitwise operations are significantly faster than arithmetic for certain operations
2. **Portability** — be careful with signed right shifts and bitfields
3. **Safety** — always check for overflow and division by zero
4. **Readability** — use parentheses to make precedence explicit

5. **Testing** — different compilers/architectures may handle undefined behavior differently

Best Practices Checklist

- [ ] Use `==` for comparison, `=` only for assignment
- [ ] Use parentheses to clarify precedence
- [ ] Check bounds before array access
- [ ] Validate division/modulo operands
- [ ] Use unsigned types for bit manipulation
- [ ] Avoid undefined behavior (no `i = i++`)
- [ ] Use `static inline` for performance-critical bit operations
- [ ] Document bit positions and masks
- [ ] Consider endianness in data serialization
- [ ] Use compiler warnings (`-Wall -Wextra`)

Additional Resources

```
Recommended Reading:
1. "Hacker's Delight" — Henry S. Warren Jr.
2. "Computer Systems: A Programmer's Perspective"
3. "Embedded C Coding Standard" — Michael Barr
4. MISRA C guidelines for embedded systems
```

Conclusion

Operators in C are not just syntactic elements — they are deeply tied to computer architecture and system-level programming. Understanding them at this level of detail allows you to write highly efficient embedded code, debug complex issues in system software, optimize performance-critical sections, understand compiler output and assembly, and work with hardware registers effectively.

> In C, you're programming close to the metal. The operators you use are often directly translated to CPU instructions, making a deep understanding of them crucial for high-performance, systems-level programming.

Part 11 — Three Classic C Programming Problems

Problem 1: Swap Two Numbers Without a Third Variable

1.1 Mathematical Methods

Method A — Addition and Subtraction

```c
#include <stdio.h>

void swap_add_sub(int *a, int *b) {
    *a = *a + *b;   // Step 1: a becomes the sum
    *b = *a - *b;   // Step 2: b becomes original a
    *a = *a - *b;   // Step 3: a becomes original b
}

int main() {
    int x = 10, y = 20;

    printf("Before swap: x = %d, y = %d\n", x, y);
    swap_add_sub(&x, &y);
    printf("After swap:  x = %d, y = %d\n", x, y);

    return 0;
}
```

Step-by-step walkthrough:

```
Initial: a = 10, b = 20
Step 1: a = a + b = 10 + 20 = 30
Step 2: b = a - b = 30 - 20 = 10   (b now has original a)
Step 3: a = a - b = 30 - 10 = 20   (a now has original b)
Final:   a = 20, b = 10  ✓
```

> ⚠ **Warning — overflow risk!** This method can overflow if `a + b > INT_MAX`.

```c
// Example on a 32-bit system:
int a = 2147483647;   // INT_MAX
int b = 100;
// a + b = 2147483747 > INT_MAX → Overflow! Undefined behavior
```

Method B — Multiplication and Division

```c
void swap_mul_div(int *a, int *b) {
    // Only works if neither value is zero
    *a = *a * *b;
    *b = *a / *b;
    *a = *a / *b;
}

// ⚠ RISKS:
// 1. Overflow: a * b may exceed INT_MAX
// 2. Division by zero if either value is 0
// 3. Precision loss with floating point
```

Method C — XOR (Bitwise) — The Best Method

```c
#include <stdio.h>

void swap_xor(int *a, int *b) {
    *a = *a ^ *b;
    *b = *a ^ *b;
    *a = *a ^ *b;
}

int main() {
    int x = 10, y = 20;

    printf("Before swap: x = %d, y = %d\n", x, y);
    swap_xor(&x, &y);
    printf("After swap:  x = %d, y = %d\n", x, y);

    return 0;
}
```

Deep dive — XOR swap visualization (8-bit example):

```
a = 10 (00001010)
b = 20 (00010100)

Step 1: a = a ^ b
  00001010 (10)
^ 00010100 (20)
= 00011110 (30)   ← new a

Step 2: b = a ^ b
  00011110 (30)
^ 00010100 (20)
= 00001010 (10)   ← b now has original a

Step 3: a = a ^ b
  00011110 (30)
^ 00001010 (10)
= 00010100 (20)   ← a now has original b

Final: a = 20, b = 10  ✓
```

XOR properties used:

```
1. Commutative:  a ^ b = b ^ a
2. Associative:  (a ^ b) ^ c = a ^ (b ^ c)
3. Identity:     a ^ 0 = a
4. Self-inverse: a ^ a = 0
5. This gives us: (a ^ b) ^ a = b  and  (a ^ b) ^ b = a
```

1.2 Complete Implementation with All Methods

```c
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>

// METHOD 1: XOR (most efficient, no overflow)
void swap_xor(int *a, int *b) {
    if (a == b) return;   // Important: handle same memory location
    *a = *a ^ *b;
    *b = *a ^ *b;
    *a = *a ^ *b;
}

// METHOD 2: Addition/Subtraction (simple but can overflow)
void swap_add_sub(int *a, int *b) {
    if (a == b) return;
    // Check for overflow
    if ((*a > 0 && *b > INT_MAX - *a) || (*a < 0 && *b < INT_MIN - *a)) {
        printf("Warning: Overflow possible! Using XOR instead.\n");
        swap_xor(a, b);
        return;
    }
    *a = *a + *b;
    *b = *a - *b;
    *a = *a - *b;
}

// METHOD 3: Multiplication/Division (least recommended)
void swap_mul_div(int *a, int *b) {
    if (a == b) return;
    if (*a == 0 || *b == 0) {
        printf("Error: Cannot swap with zero using multiplication method\n");
        return;
    }
    *a = *a * *b;
    *b = *a / *b;
    *a = *a / *b;
}

// METHOD 4: Using a temporary variable (most readable)
void swap_temp(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Demonstration function
void demonstrate_swap() {
    int a, b;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    printf("\nOriginal values: a = %d, b = %d\n", a, b);

    // Test XOR swap
    int x = a, y = b;
    swap_xor(&x, &y);
    printf("XOR Swap:       a = %d, b = %d\n", x, y);

    // Test Add/Sub swap
    x = a, y = b;
    swap_add_sub(&x, &y);
    printf("Add/Sub Swap:   a = %d, b = %d\n", x, y);

    // Test Temp swap
    x = a, y = b;
    swap_temp(&x, &y);
    printf("Temp Swap:      a = %d, b = %d\n", x, y);
}

int main() {
```

```
    demonstrate_swap();
    return 0;
}
```

1.3 Advanced: Swapping Without a Function Call

```
// Using a macro (inline expansion)
#define SWAP_XOR(a, b) do { \
    (a) ^= (b);             \
    (b) ^= (a);             \
    (a) ^= (b);             \
} while(0)

// Even safer macro with type checking
#define SWAP_GENERIC(a, b, type) do { \
    type temp = (a);                  \
    (a) = (b);                        \
    (b) = temp;                       \
} while(0)

int main() {
    int x = 5, y = 10;
    float p = 3.14, q = 2.71;

    SWAP_XOR(x, y);
    SWAP_GENERIC(p, q, float);

    printf("x=%d, y=%d\n", x, y);      // x=10, y=5
    printf("p=%.2f, q=%.2f\n", p, q);  // p=2.71, q=3.14

    return 0;
}
```

Problem 2: Check Even/Odd Using a Bitwise Operator

2.1 The Mathematical Foundation

```
Binary Representation:
Even numbers have LSB (Least Significant Bit) = 0
Odd numbers have LSB (Least Significant Bit) = 1

Example:
5 (00000101) → LSB = 1 → ODD
6 (00000110) → LSB = 0 → EVEN
```

2.2 Basic Implementation

```c
#include <stdio.h>
#include <stdbool.h>

// Method 1: direct bit check
bool is_even_bitwise(int n) {
    return !(n & 1);   // Check LSB
}

bool is_odd_bitwise(int n) {
    return n & 1;       // Check LSB
}

int main() {
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);

    // Using bitwise operator
    if (num & 1) {
        printf("%d is ODD (LSB = 1)\n", num);
    } else {
        printf("%d is EVEN (LSB = 0)\n", num);
    }

    return 0;
}
```

2.3 Comprehensive Implementation

```c
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>

// Method 1: using AND with 1 (most efficient)
bool is_even_and(int n) {
    return !(n & 1);
}

// Method 2: using XOR (alternative)
bool is_even_xor(int n) {
    return (n ^ 1) > n;   // XOR with 1 increases odd numbers
}

// Method 3: using division (slow, not recommended)
bool is_even_division(int n) {
    return (n / 2) * 2 == n;   // Check if divisible by 2
}

// Method 4: using modulo (simple but slower than bitwise)
bool is_even_modulo(int n) {
    return n % 2 == 0;
}

// Method 5: using shift (alternative)
bool is_even_shift(int n) {
    return (n >> 1) << 1 == n;   // Shift right, then left
}

// Advanced: check if multiple of any power of 2
bool is_multiple_of_power_of_two(int n, int power) {
    // Check if n is divisible by 2^power
    return (n & ((1 << power) - 1)) == 0;
}
```

2.4 Visual Explanation

```c
#include <stdio.h>

void print_binary(int n, int bits) {
    for (int i = bits - 1; i >= 0; i--) {
        printf("%d", (n >> i) & 1);
        if (i % 4 == 0 && i != 0) printf(" ");
    }
    printf("\n");
}

void analyze_number(int n) {
    printf("\n--- Analysis of %d ---\n", n);
    printf("Binary: ");
    print_binary(n, 16);
    printf("LSB:    %d\n", n & 1);

    if (n & 1) {
        printf("Result: ODD number\n");
        printf("Bit 0 is set to 1\n");
    } else {
        printf("Result: EVEN number\n");
        printf("Bit 0 is set to 0\n");
    }
}

int main() {
    int numbers[] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
    for (int i = 0; i < 11; i++) {
        analyze_number(numbers[i]);
    }
    return 0;
}
```

2.5 Advanced Applications

```c
// 1. Count odd numbers in an array
int count_odd(int arr[], int size) {
    int count = 0;
    for (int i = 0; i < size; i++) {
        count += (arr[i] & 1);   // Adds 1 if odd, 0 if even
    }
    return count;
}

// 2. Check parity of binary representation
bool has_odd_parity(int n) {
    int parity = 0;
    while (n) {
        parity ^= (n & 1);   // XOR all bits
        n >>= 1;
    }
    return parity;   // 1 if odd number of 1s
}

// 3. Efficient parity check (Brian Kernighan style)
bool has_odd_parity_fast(unsigned int n) {
    n ^= n >> 16;
    n ^= n >> 8;
    n ^= n >> 4;
    n ^= n >> 2;
    n ^= n >> 1;
    return n & 1;
}

// 4. Round down to nearest even
int round_down_even(int n) {
    return n & ~1;   // Clear LSB
}

// 5. Round up to nearest even
int round_up_even(int n) {
    return (n + 1) & ~1;
}

// 6. Get next odd/even number
int next_odd(int n) {
    return (n & 1) ? n + 2 : n + 1;
}

int next_even(int n) {
    return (n & 1) ? n + 1 : n + 2;
}
```

Problem 3: Find the Largest of Three Numbers

3.1 Basic Approaches

Method 1 — Nested If-Else (readable)

```c
#include <stdio.h>

int find_largest_if_else(int a, int b, int c) {
    if (a >= b) {
        if (a >= c)
            return a;
        else
            return c;
    } else {
        if (b >= c)
            return b;
        else
            return c;
    }
}
```

Method 2 — Logical AND (clean)

```c
int find_largest_logical(int a, int b, int c) {
    if (a >= b && a >= c)
        return a;
    else if (b >= a && b >= c)
        return b;
    else
        return c;
}
```

3.2 Advanced Implementations

```c
#include <stdio.h>
#include <limits.h>

// Method 3: ternary operator (concise)
int find_largest_ternary(int a, int b, int c) {
    return (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c);
}

// Method 4: using a temporary variable
int find_largest_temp(int a, int b, int c) {
    int max = a;
    if (b > max) max = b;
    if (c > max) max = c;
    return max;
}

// Method 5: incremental comparison
int find_largest_math(int a, int b, int c) {
    int max = a;
    max = (b > max) ? b : max;
    max = (c > max) ? c : max;
    return max;
}

// Method 6: bit manipulation (no branching)
int find_largest_bitwise(int a, int b, int c) {
    // Find max of a and b using bit operations
    int ab_max = a ^ ((a ^ b) & -(a < b));
    // Find max of ab_max and c
    return ab_max ^ ((ab_max ^ c) & -(ab_max < c));
}

// Method 7: using macros
#define MAX(a, b) (((a) > (b)) ? (a) : (b))
#define MAX3(a, b, c) MAX(MAX(a, b), c)

// Method 8: finding maximum and minimum simultaneously
void find_max_min(int a, int b, int c, int *max, int *min) {
    *max = a;
    *min = a;

    if (b > *max) *max = b;
    if (c > *max) *max = c;

    if (b < *min) *min = b;
    if (c < *min) *min = c;
}
```

3.3 Robust Implementation with Full Statistics

```c
#include <stdio.h>
#include <limits.h>
#include <stdbool.h>

// Structure to return multiple values
typedef struct {
    int max;
    int min;
    int second_max;
} Result;

// Find all statistics of three numbers
Result analyze_three_numbers(int a, int b, int c) {
    Result result;

    result.max = a;
    result.min = a;
    result.second_max = INT_MIN;

    if (b > result.max) {
        result.second_max = result.max;
        result.max = b;
    } else if (b > result.second_max) {
        result.second_max = b;
    }

    if (c > result.max) {
        result.second_max = result.max;
        result.max = c;
    } else if (c > result.second_max) {
        result.second_max = c;
    }

    if (b < result.min) result.min = b;
    if (c < result.min) result.min = c;

    return result;
}
```

3.4 Advanced: Generic Type-Safe Macros

```c
#include <stdio.h>
#include <stddef.h>

// Type-safe max macro with typeof (GCC extension)
#define MAX_SAFE(a, b) ({ \
    typeof(a) _a = (a); \
    typeof(b) _b = (b); \
    _a > _b ? _a : _b; \
})

#define MAX3_SAFE(a, b, c) MAX_SAFE(MAX_SAFE(a, b), c)

// For C11 with _Generic
#define MAX_GENERIC(a, b) _Generic((a), \
    int: max_int, \
    float: max_float, \
    double: max_double \
)(a, b)

int max_int(int a, int b) { return a > b ? a : b; }
float max_float(float a, float b) { return a > b ? a : b; }
double max_double(double a, double b) { return a > b ? a : b; }

int main() {
    int i1 = 10, i2 = 20, i3 = 15;
    float f1 = 3.14, f2 = 2.71, f3 = 4.20;

    printf("Max int: %d\n", MAX3_SAFE(i1, i2, i3));
    printf("Max float: %.2f\n", MAX3_SAFE(f1, f2, f3));

    printf("Generic max int: %d\n", MAX_GENERIC(10, 20));
    printf("Generic max float: %.2f\n", MAX_GENERIC(3.14f, 2.71f));

    return 0;
}
```

Combined Demonstration Program

A single menu-driven program exercising all three problems together, including binary display of swap results.

```c
#include <stdio.h>
#include <stdbool.h>
#include <limits.h>

// ================ SWAP FUNCTIONS ================
void swap_xor(int *a, int *b) {
    if (a == b) return;
    *a ^= *b;
    *b ^= *a;
    *a ^= *b;
}

void swap_add_sub(int *a, int *b) {
    if (a == b) return;
    *a += *b;
    *b = *a - *b;
    *a -= *b;
}

// ================ EVEN/ODD CHECK ================
bool is_even(int n) { return !(n & 1); }
bool is_odd(int n)  { return n & 1; }

// ================ MAX FINDER ================
int max_three(int a, int b, int c) {
    return (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c);
}

int main() {
    int choice;

    do {
        printf("\n========================================");
        printf("\n      C OPERATORS - DEMO PROGRAM");
        printf("\n========================================");
        printf("\n1. Swap two numbers");
        printf("\n2. Check even/odd using bitwise");
        printf("\n3. Find largest of three numbers");
        printf("\n4. Run ALL demonstrations");
        printf("\n0. Exit");
        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1: /* demonstrate_swap(); */ break;
            case 2: /* demonstrate_even_odd(); */ break;
            case 3: /* demonstrate_max(); */ break;
            case 4: /* run all three */ break;
            case 0: printf("\nThank you for using the program!\n"); break;
            default: printf("\nInvalid choice! Please try again.\n");
        }
    } while (choice != 0);

    return 0;
}
```

Performance Comparison

Benchmarking each technique over 10,000,000 iterations, using `clock()` from `<time.h>` and a `volatile` accumulator to prevent compiler over-optimization:

```c
#include <stdio.h>
#include <time.h>
#include <stdlib.h>

#define ITERATIONS 10000000

void performance_comparison() {
    clock_t start, end;
    double cpu_time_used;
    volatile int result = 0;   // Prevent optimization

    // --- Even/Odd Check ---
    start = clock();
    for (int i = 0; i < ITERATIONS; i++) result += (i & 1);   // Bitwise
    end = clock();
    printf("Bitwise: %f s\n", (double)(end - start) / CLOCKS_PER_SEC);

    start = clock();
    result = 0;
    for (int i = 0; i < ITERATIONS; i++) result += (i % 2);   // Modulo
    end = clock();
    printf("Modulo:  %f s\n", (double)(end - start) / CLOCKS_PER_SEC);

    // --- Swap ---
    int x = 5, y = 10;
    start = clock();
    for (int i = 0; i < ITERATIONS; i++) { x ^= y; y ^= x; x ^= y; }
    end = clock();
    printf("XOR swap:  %f s\n", (double)(end - start) / CLOCKS_PER_SEC);

    start = clock();
    for (int i = 0; i < ITERATIONS; i++) { int temp = x; x = y; y = temp; }
    end = clock();
    printf("Temp swap: %f s\n", (double)(end - start) / CLOCKS_PER_SEC);
}
```

Summary and Best Practices

Quick Reference

Problem	Best Method	Why	Alternative
Swap	XOR	No overflow, fastest, no temp variable	Temp variable (readable)
Even/Odd	`n & 1`	Fastest, direct bit operation	`n % 2` (readable)
Max of 3	Ternary	Concise, no branching	If-else (clear)

Best Practices Checklist

- [ ] **Swap:** use XOR for performance-critical code, temp variable for readability
- [ ] **Even/Odd:** always use `n & 1` in performance-critical code
- [ ] **Max:** use the ternary operator for concise code, if-else for clarity
- [ ] **Edge cases:** handle zero, negative numbers, and overflow
- [ ] **Documentation:** comment bitwise operations clearly
- [ ] **Testing:** test with boundary values (`INT_MAX` , `INT_MIN`)

Common Mistakes to Avoid

```
//    WRONG: swapping the same variable
swap_xor(&x, &x);   // Results in x = 0

//    CORRECT: check and handle
if (a != b) swap_xor(&a, &b);

//    WRONG: using modulo for performance-critical checks
if (n % 2 == 0)     // Slow

//    CORRECT: using bitwise
if (!(n & 1))        // Fast

//    WRONG: overflow in addition-based swap
*a = *a + *b;         // May overflow

//    CORRECT: use XOR for a safe swap
*a ^= *b;
*b ^= *a;
*a ^= *b;
```

Final Word

These three classic problems demonstrate the power of C operators: the **XOR swap** shows the elegance of bitwise operations, the **even/odd check** demonstrates hardware-level efficiency, and the **maximum finder** showcases decision-making with operators.

> The best code is not just efficient but also maintainable — use these techniques wisely.